

第1章 计算机系统概述

- [计算机发展历程与趋势](#)
- [体系结构的伟大思想](#)
- [计算机层次结构](#)
 - [系统层次（由低到高）](#)
 - [冯·诺依曼架构](#)
 - [核心特点](#)
 - [架构改进：哈佛架构](#)
 - [架构改进：以存储器为中心的理念](#)
- [计算机性能指标](#)
 - [非时间相关指标](#)
 - [时间相关指标](#)
 - [例题](#)

/

计算机发展历程与趋势

发展历程:

电子管 → 晶体管 → 集成电路 → 大规模集成电路

发展趋势:

体积减小、速度提高、价格下降、功耗降低、可靠性增强

体系结构的伟大思想

1. 使用抽象简化设计：通过接口和模块降低系统复杂度。
2. 加速常见情况：优化高频路径，提高平均性能。
3. 并行性：多个部件/任务同时运行，提高处理能力。
4. 流水线：将任务分阶段执行，实现指令级并行。
5. 预测：通过预测分支、缓存等提高执行效率。
6. 存储层次结构：利用 Cache 提高访问速度。
7. 容错与冗余：增加可靠性，如奇偶校验、ECC。

计算机层次结构

系统层次（由低到高）

- 器件层（晶体管）
- 电路层（逻辑门）
- 微结构层（数据通路）←
- 指令集层（e.g. RISC-V）←
- 操作系统层
- 程序设计语言层
- 应用层

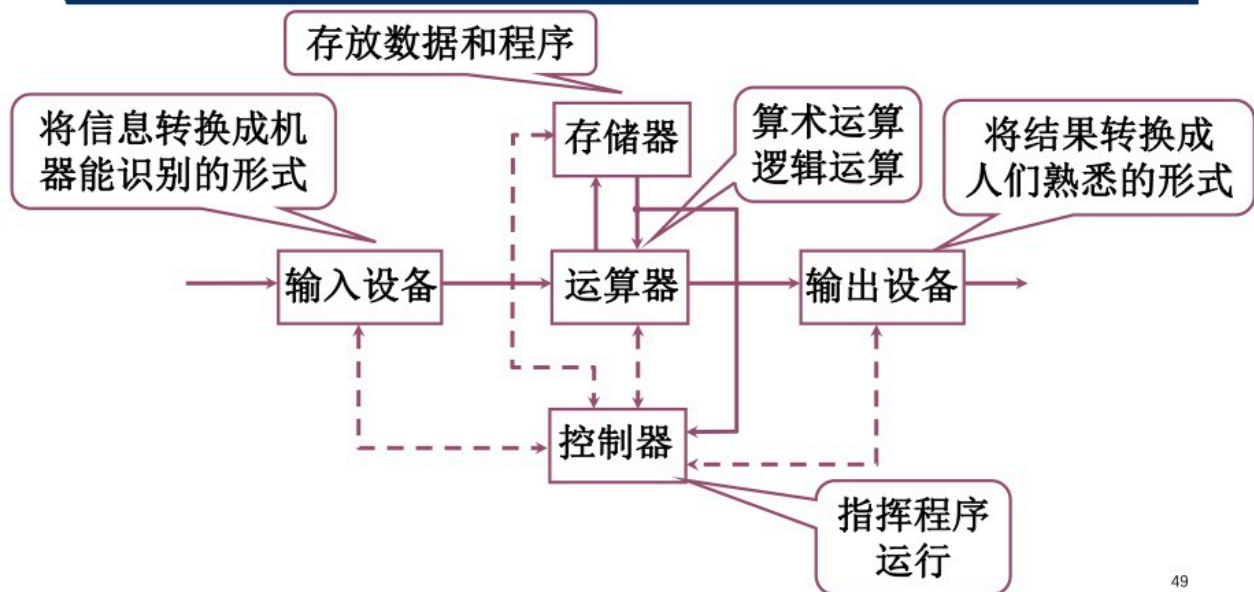
使用逐层抽象的方法降低开发计算机这一复杂系统的难度，提高稳定性和开发效率。

冯·诺依曼架构

核心特点

- 五大组件：输入、输出、控制器、运算器、存储器；
- **以运算器为中心**：早期架构中，数据流动大多需要经过运算器。
- **程序和数据混存在内存中**，按照地址访存；
- 指令和数据用二进制表示（废话）；
- 指令由操作码和地址码组成：操作码指示指令的位置，地址码指示操作数的位置；
- 指令顺序存放、顺序执行，特定条件下可以改变执行顺序（通过跳转指令）；

冯·诺依曼架构计算机硬件示意图

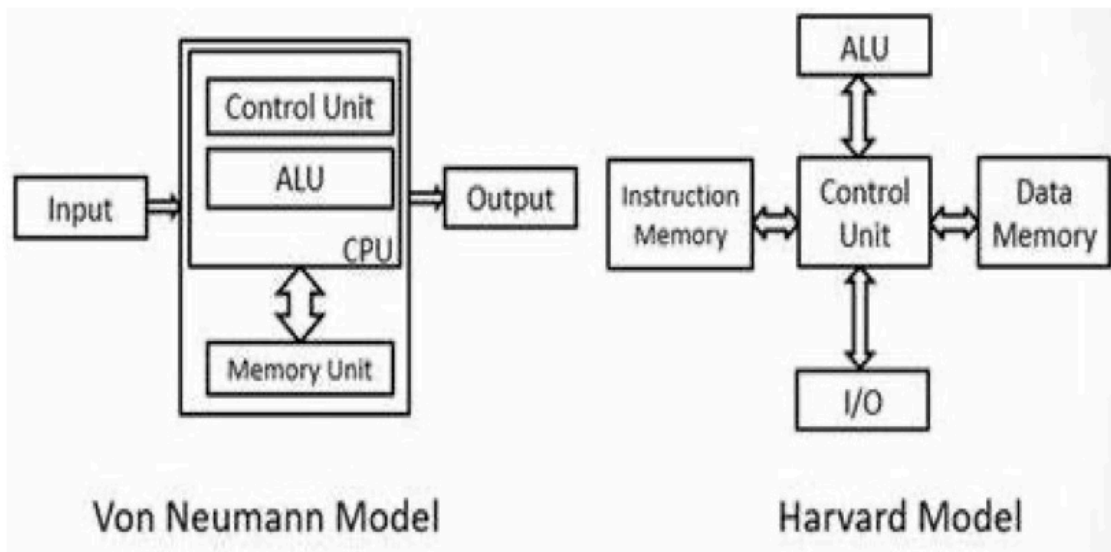


架构改进：哈佛架构

哈佛结构将程序和数据分开储存在不同的内存段中，并且分开了地址总线 and 数据总线，因此允许同时取指和取操作数，改进了并行程度。

改进型的哈佛架构（如 ARM）仍然数据、指令分开，但是复用总线。现代 CPU 在外部总线上看起来仍是冯·诺依曼架构，但是由于 CPU 内部普遍设计了 L1-L3 三层高速缓存，实际上内部来看已经类似改进型的哈佛结构。

哈佛架构 vs. 冯诺依曼架构

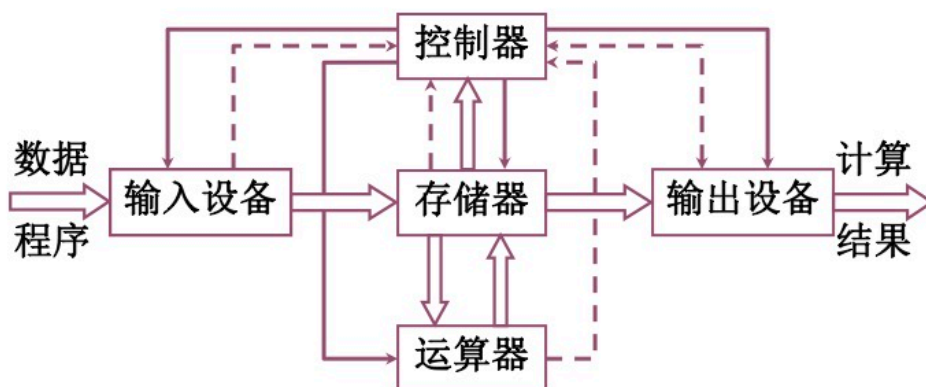


架构改进：以存储器为中心的理念

早期冯·诺依曼架构的特点之一：运算器是数据流动的核心。这导致运算器成为数据流动的瓶颈。现代计算机的组织方式中，往往是存储器作为连接CPU、输入设备和输出设备的中心（见下图）。

哈佛架构和该理念类似，通过分离指令和数据存储来优化对存储器的访问，强调访问存储器的效率。

以存储器为中心的计算机硬件示意图



计算机性能指标

非时间相关指标

- 机器字长：CPU 一次能处理的数据位数（常见 32-bit、64-bit），一般与内部寄存器的位数相等；
- 总线宽度：数据 / 地址总线一次能并行传送的最大信息位数，也称为位宽；
- 主存容量：内存总量，等于存储单元总数 $\times$ 存储字长，单位为 Byte（现代内存常用 GB）；
- 主存带宽：内存单位时间传输能力，单位为 Byte/s。

时间相关指标

- 程序（Program）：一系列指令的集合；
- 指令（Instruction）：CPU 能直接理解执行的低级操作，如算术运算、条件判断、跳转等，不同的指令耗费时钟周期的数量不同；
- 时钟周期（Clock Cycle）：单个 CPU 硬件层面的最基本操作所需的时间，单位 s 或 ns；

这是一些基本概念，下面的概念都是上面三个概念的数目和所需时间之间的简单运算。

- 主频（Frequency）：CPU 单位时间内完成时钟周期的数量，单位 Hz（现代 CPU 常用 GHz）；

时钟周期和主频互为倒数

$$\text{Clock Cycle} \times \text{Frequency} = 1$$

- CPI: Cycle Per Instruction，执行某个程序时，每条指令平均所需的时钟周期数，与构成程序的指令复杂程度有关。

$$\text{CPI} = \frac{\sum_{\text{Instructions}} \text{Clock Cycle per Instruction}}{\text{Number of Instructions}}$$

- IPC: Instructions Per (Clock) Cycle，每时钟周期能执行的指令数量，小于 1；

$$\text{CPI} \times \text{IPC} = 1$$

- IPS: Instructions Per Second，每秒能执行的指令数量；
- MIPS: Million Instructions Per Second，百万指令数量每秒；
- MFLOPS: Million Floating-point Operations Per Second，百万浮点操作数每秒，与上面的 MIPS 类似；

$$\begin{aligned} \text{IPS} &= \text{IPC} \times \text{Frequency} \\ \text{MIPS} &= \text{IPS} \times 10^{-6} \end{aligned}$$

- CPU 执行时间：也可以简称 CPU 时间，是指一个程序在 CPU 上所花费的时间；
 - 不包括等待 I/O 或运行其他程序的时间；
 - 用户CPU时间：程序本身所花的CPU时间；
 - 系统CPU时间：为执行程序而花费在操作系统上的时间；
- 响应时间：计算机完成某个任务所需的总时间，包括硬盘访问、内存访问、I/O 活动、操作系统开销和 CPU 执行时间等；
- 吞吐率：也叫带宽单位时间内完成的任务数（如请求数、运算数）；

例题

例子1：计算CPI和MIPS

- 假设一台计算机主频 f 为**1GHZ**，在其上运行由 **2×10^5** 条指令组成的目标代码，程序主要由**4**类指令组成，他们所占的比例和各自的**CPI**如下表所示，求程序的**CPI**和**MIPS**。

指令类型	CPI	指令比例
算术和逻辑	1	60%
Load/Store	2	18%
转移	4	12%
Cache缺失访存	8	10%

解： $CPI = 1 \times 60\% + 2 \times 18\% + 4 \times 12\% + 8 \times 10\% = 2.24$

$$MIPS = (f/CPI)/10^6 = 1 \times 10^9 / 2.24/10^6 = 446.4$$

62

程序的 CPI 即程序中每条指令平均所需的时钟周期数，对该程序中不同类型的指令的 CPI 加权平均即可。

根据上述笔记中的式子，组合后可以得到：

$$\begin{aligned}
 MIPS &= IPS \times 10^{-6} \\
 &= IPC \times \text{Frequency} \times 10^{-6} \\
 &= CPI^{-1} \times \text{Frequency} \times 10^{-6}
 \end{aligned}$$

假设一台计算机主频 f 为1GHZ，在其上运行由 2×10^5 条指令组成的目标代码，程序主要由4类指令组成，他们所占的比例和各自的CPI如下表所示，求程序执行时间。

- ☐ A 0.25ms
- ☒ B 0.45ms
- ☐ C 0.2ms
- ☐ D 0.35ms
- ☐ E 以上都不对

指令类型	CPI	指令比例
算术和逻辑	1	60%
Load/Store	2	18%
转移	4	12%
Cache缺失访存	8	10%

CPI = 2.24 MIPS = 446.4

提交

63

$$\begin{aligned}
 \text{CPU时间} &= \text{Number of Instructions} \times \text{IPS}^{-1} \\
 &= \text{Number of Instructions} \times (\text{IPC} \times \text{Frequency})^{-1} \\
 &= \text{Number of Instructions} \times \text{CPI} \times \text{Frequency}^{-1}
 \end{aligned}$$

或者

$$\begin{aligned}
 \text{CPU时间} &= \text{Number of Instructions} \times \text{IPS}^{-1} \\
 &= \text{Number of Instructions} \times (\text{MIPS} \times 10^6)^{-1}
 \end{aligned}$$